

click on the *Add* button.

4. The *Add Repository* dialogue box will open. Give any related name to the repository, and in the location textbox copy the above URL again and click *OK*.
5. Click on *Select All* to select all the components and click *Next*.
6. Again click on *Next* on the next window of the dialogue box.
7. The *Review License* window will open. Click the *I accept terms...* radio button or you can also go through the licence agreement if you wish to. Now click the *Finish* button.
8. It will take some time to install the plugin.
9. After the progress bar shows that Eclipse has completed its installation of the plugin, it will ask you to restart the Eclipse IDE. Click *Yes* to restart it.

After Eclipse has been restarted, you can verify your installation by creating a new JavaFX project. For this, go to *File > New > Project* and scroll down to the JavaFX option available in the *New Project* dialogue box. Expand it, select the JavaFX project and click *Next*. Give your project an appropriate name, click *Next* and then *Finish*. Now you can view a newly created JavaFX project as shown in Figure 2 and you can start working on it.

After having installed the JavaFX plugin and JAR in the Eclipse IDE, we will start off with the coding of our first JavaFX application. This will be a basic GUI with a textbox that will accept any text and a button, which on clicking, will print whatever text you have entered in the textbox. I know it's a pretty basic implementation, but this will help you to understand how the code works and the different components necessary for the development of multi-tier applications.

Coding your first JavaFX GUI application

I know you must be excited to code your first JavaFX application. You probably know by now that there are two ways to create a project. The first way is to create a normal Java project and associate the necessary JARs with it. The second way is to directly create a JavaFX project. We will use the second method in this tutorial but to develop your application, you can choose any method, based on your convenience. To start off, open the Eclipse IDE, set an appropriate workspace and create a new JavaFX project as mentioned above.

1. After you have created your project, you can view it in your *Package Explorer*. Expand your project and you will be able to view the *src* folder and other necessary files, like the JRE system library and the JavaFX SDK attached to it.
2. Expand the *src* folder and delete the auto-generated package application by choosing the *Delete* option and right-clicking on it.
3. Now right-click on the *src* folder and add one package to it.
4. After your defined package has been created, create a new *driverClass.java* class under that package. Now we can start with the code.
5. Write the code given below in your Java class. This code is without exception handling and I will explain it

after running it.

```
import javafx.application.Application;
import javafx.event.ActionEvent;
import javafx.event.EventHandler;
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.TextField;
import javafx.scene.layout.BorderPane;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class driverClass extends Application{

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        launch(args);
    }

    @Override
    public void start(Stage primaryStage) throws Exception
    {
        // TODO Auto-generated method stub
        primaryStage.setTitle("Open Source For You - First
program");
        TextField textField = new TextField();
        Button btn = new Button("Click me to reveal the
above text");
        btn.setOnAction(new EventHandler<ActionEvent>() {

            @Override
            public void handle(ActionEvent event) {
                // TODO Auto-generated method stub
                System.out.println("Entered text is " +
                    textField.getText());

                textField.clear();
            }
        });
        BorderPane pane = new BorderPane();
        pane.setPadding(new Insets(70));
        VBox paneCenter = new VBox();
        paneCenter.setSpacing(10);
        pane.setCenter(paneCenter);
        paneCenter.getChildren().add(textField);
        paneCenter.getChildren().add(btn);
        Scene scene= new Scene(pane, 400, 200);
        primaryStage.setScene(scene);
        primaryStage.show();
    }
}
```

Continued on Page 43...